Heuristic search: memory usage



Zola Mahlaza

Department of Computer Science

Feb 2026

Topics

- A* search: space complexity
- Beam search

A* search

- Cost optimality of A* search depends on heuristic
- If heuristic is admissible and consistent then A* search is optimal and complete (assuming finite search space)
- A* is complete and optimal but…
  - The space complexity of A* is $O(b^d)$. b = branching factor, d = depth

Sources of memory usage

- There are two sources of memory consumption:
  - Openlist (frontier) and closed list (reached)

Sources of memory usage: Reference counts

- There are two sources of memory consumption:
  - Openlist (frontier) and closed list (reached)
- A* stores every expanded node in the closed list to prevent cycles and redundant work
- In large state spaces, list consumes all available RAM
- For each node, keep a reference count (e.g., 3)
- A node can be permanently removed from closed list once count reaches zero

Beam Search

1. sort by f
2. select only top k elements

```
function BEST-F_ T-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(ST TE=problem.INITIAL)
    frontier ← a priority queue ordered by f, with node as an element
    reached ← a lookup table, with one entry with key problem.INITIAL and value node
    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s is not in reached                                  then
                reached[s] ← child
                add child to frontier
    return failure
```

Beam search example

- Consider an autonomous agent whose goal is to move from (0,0) to (3,3). Beam Width (k=2)
- Actions {Up, Down, Left, Right}
- Heuristic is Manhattan distance

start			
			goal

Beam search example

(0,0), h = 6

start			
			goal

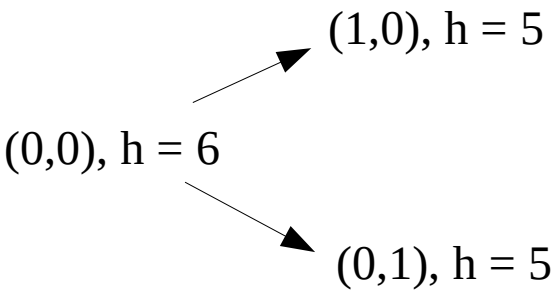
Open

(0,0)

Closed

(0,0)

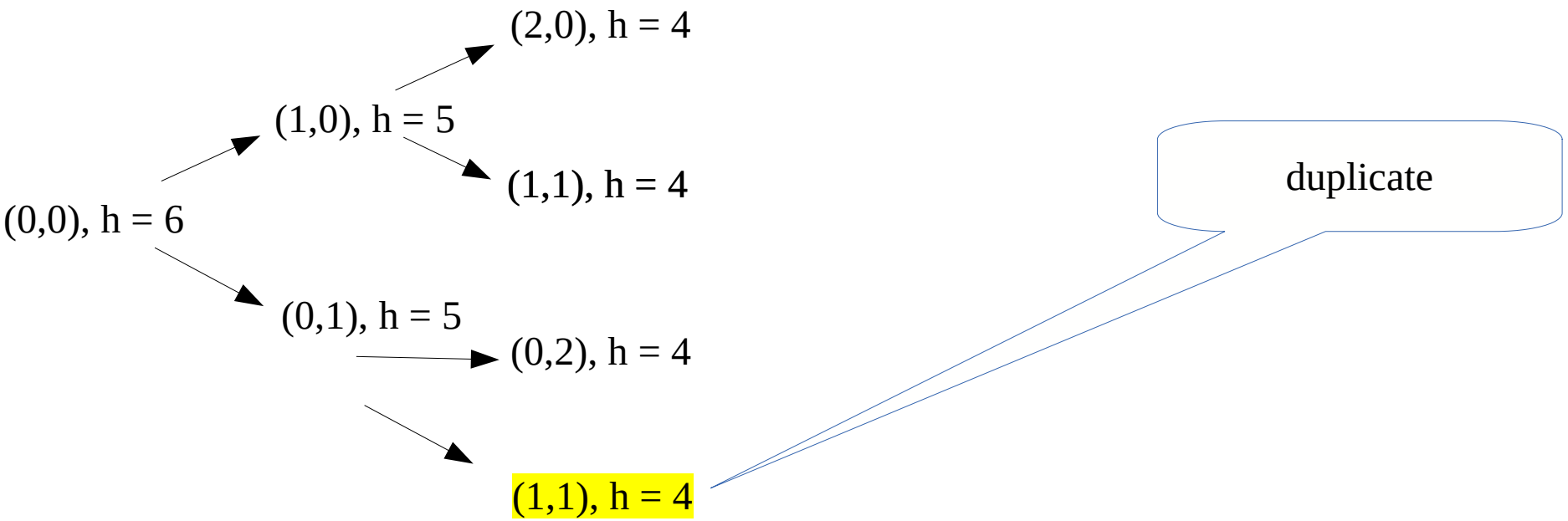
Beam search example



Open	Closed
(1,0) (0,1)	(1,0) (0,1) (0,0)

start	?		
?			
			goal

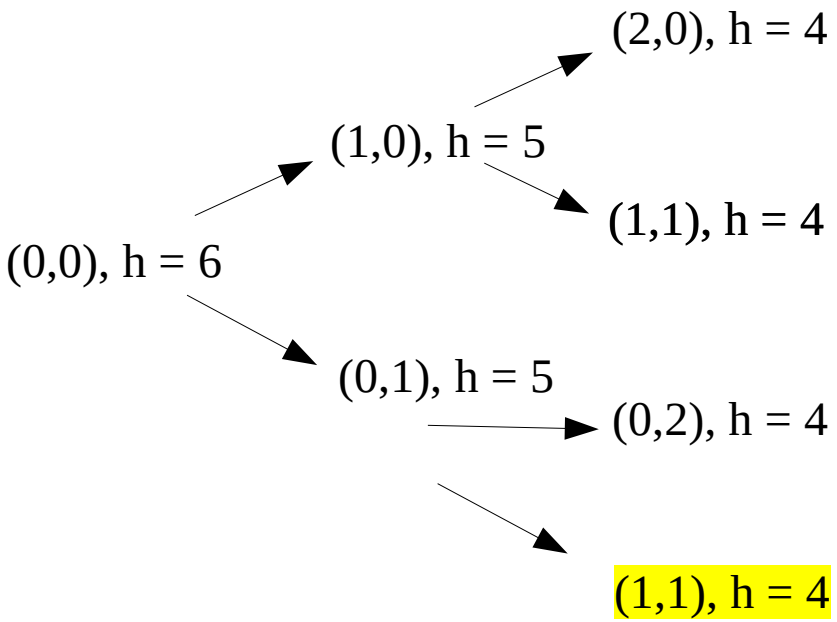
Beam search example

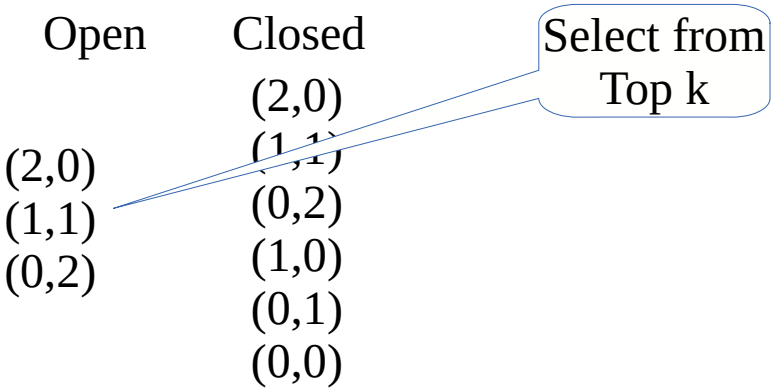


Open	Closed
	(2,0)
(2,0)	(1,1)
(1,1)	(0,2)
(0,2)	(1,0)
	(0,1)
	(0,0)

start		?	
	?		
?			
			goal

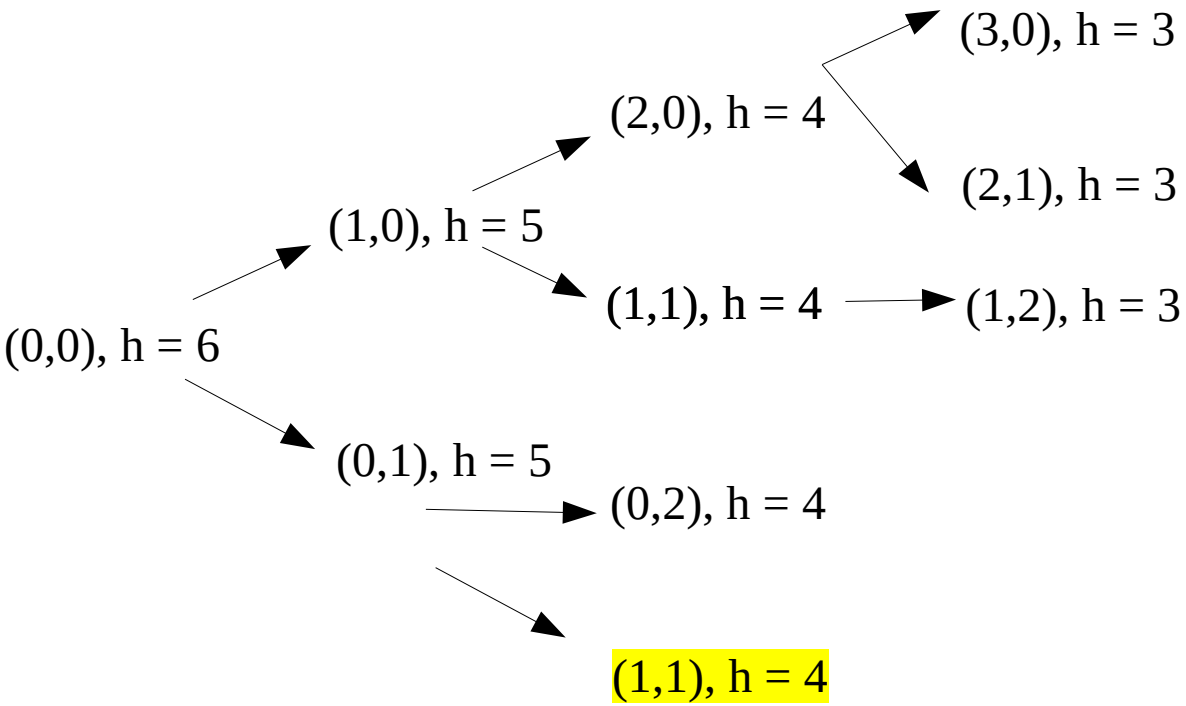
Beam search example





start		?	
	?		
?			
			goal

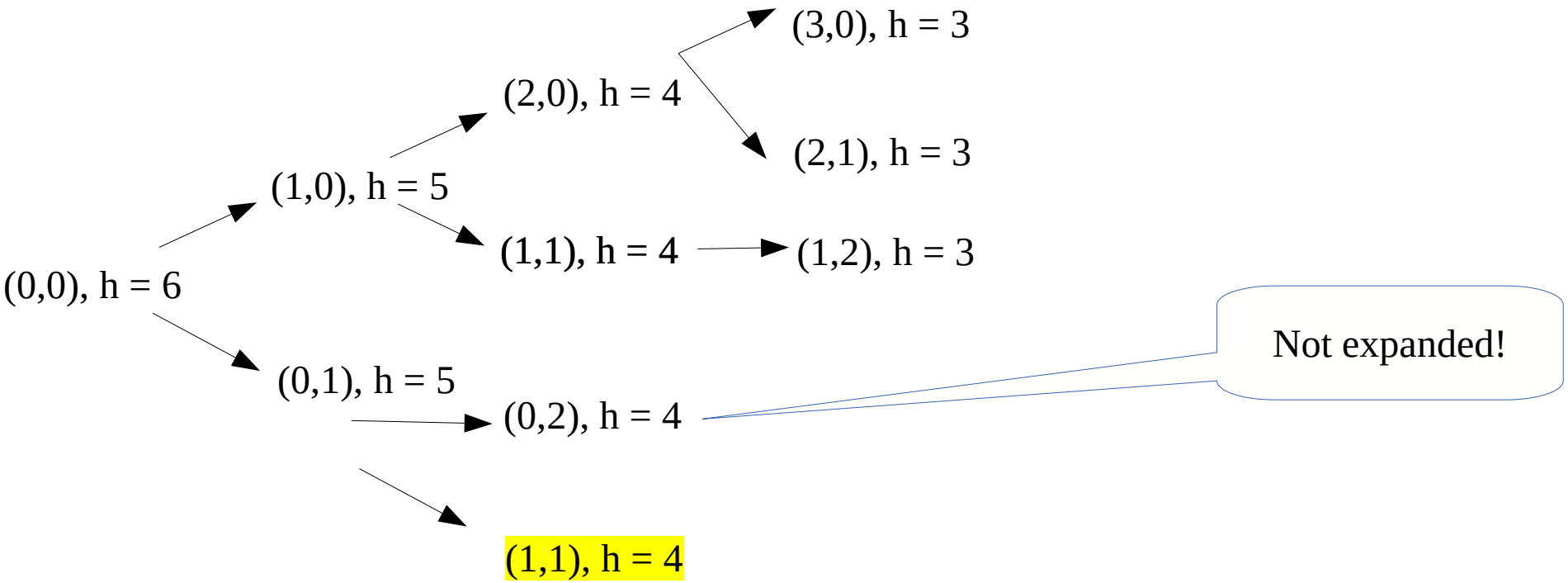
Beam search example



Open	Closed
	(2,0)
(3,0)	(1,1)
(1,2)	(0,2)
(0,2)	(1,0)
	(0,1)
	(0,0)

start			?
		?	
?	?		
			goal

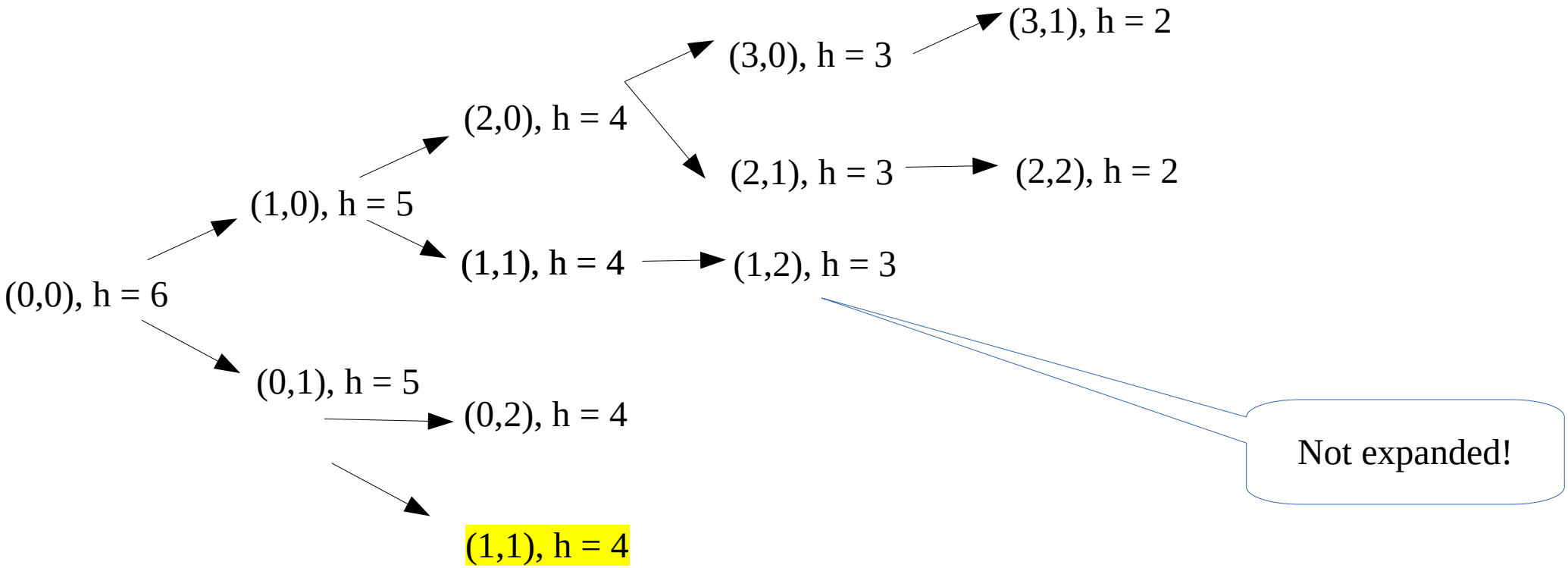
Beam search example



Open	Closed
(3,0)	(3,0)
(2,1)	(2,1)
(1,2)	(1,2)
(0,2)	(2,0)
	(1,1)
	(0,2)
	...

start			?
		?	
?	?		
			goal

Beam search example



Open	Closed
(3,1)	(3,1)
(2,2)	(2,2)
(1,2)	(3,0)
(0,2)	(2,1)
	(1,2)
	(2,0)
	...

start			
			?
		?	
			goal

Delta-beam Search

1. sort by f
2. select only elements that meet delta threshold

```
function BEST-F[illegible]T-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(ST[illegible]TE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached [illegible] then
        reached[s] ← child
        add child to frontier
  return failure
```

Traditional and delta beam search

- In traditional beam search, only the k nodes with the best heuristic scores are considered
- Delta beam search keeps any node n whose score is within a specific distance from the best node
- Delta search is more robust but may require more memory since beam width since variable
  - Depending on problem, it may exceed memory limits

A* and beam search

- A* is complete and optimal but…
  - The space complexity of A* is $O(b^d)$. b = branching factor, d = depth of goal
- Beam search is not complete or optimal but…
  - The space complexity is O(km). m = max . depth

Reading

- Sections 3.5 - 3.6.2

